

Transformer আর্কিটেকচার

সম্পূর্ণ গাণিতিক ও নিউমেরিক্যাল ওয়াকথ্রু

ভূমিকা

এই নোটটি Transformer আর্কিটেকচারকে একদম শুরু থেকে — কেন Transformer তৈরি করা হয়েছিল, তা থেকে শুরু করে — শেষ পর্যন্ত, অর্থাৎ চূড়ান্ত Output Probability এবং Next-Token Prediction পর্যন্ত সম্পূর্ণভাবে ব্যাখ্যা করবে।

এই ডকুমেন্টের মূল লক্ষ্য হলো প্রতিটি গুরুত্বপূর্ণ ধাপের সম্পূর্ণ গাণিতিক হিসাব (Full Numerical Calculation) দেখানো। কোনো ধাপেই শুধু concept বলে ছেড়ে দেওয়া হবে না — প্রতিটি Matrix Multiplication, প্রতিটি Vector, প্রতিটি Probability সরাসরি সংখ্যা দিয়ে দেখানো হবে।

পুরো ডকুমেন্টে একটিমাত্র উদাহরণ বাক্য ব্যবহার করা হবে: "I am trying to learn Machine Learning"। এই একই বাক্য Input থেকে শুরু করে Encoder, Decoder এবং চূড়ান্ত Output Probability পর্যন্ত ব্যবহৃত হবে, যাতে পাঠক প্রতিটি সংখ্যার উৎস স্পষ্টভাবে বুঝতে পারেন।

বিঃদ্রঃ এই ডকুমেন্টে ব্যবহৃত সব Weight Matrix (যেমন W_Q , W_K , W_V , W_1 , W_2 ইত্যাদি) ছোট, কাল্পনিক (illustrative) সংখ্যা — এগুলো কোনো প্রকৃত Pretrained Transformer-এর আসল Weight নয়। এগুলো শুধুমাত্র হাতে-কলমে (manually) গণনা করা সম্ভব করার জন্য বেছে নেওয়া হয়েছে।

অংশ ১: Transformer কেন তৈরি করা হলো

১. RNN (Recurrent Neural Network)

Transformer আসার আগে sequence (যেমন বাক্য) প্রসেস করার জন্য প্রধান স্থাপত্য ছিল RNN। RNN একটি sequence-কে token-by-token (একটির পর একটি) প্রসেস করে, এবং প্রতিটি ধাপে একটি Hidden State তৈরি করে যা এখন পর্যন্ত দেখা সব তথ্যের সারমর্ম (summary) ধরে রাখে।

ধরা যাক আমাদের একটি ছোট বাংলা অক্ষরের sequence আছে: ক (ka) → খ (kha) → গ (ga)। RNN প্রথমে ক পড়ে একটি Hidden State h_1 তৈরি করে। এরপর খ এবং পূর্ববর্তী h_1 ব্যবহার করে নতুন Hidden State h_2 তৈরি করে। এভাবে প্রতিটি ধাপ আগের ধাপের উপর নির্ভরশীল — অর্থাৎ প্রসেসিং সম্পূর্ণভাবে Sequential (ক্রমিক)।

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

এখানে x_t হলো t -তম Input, h_{t-1} হলো আগের Hidden State, W_x ও W_h হলো Weight Matrix, b হলো Bias, এবং f একটি Non-linear Activation Function (যেমন $\tanh$)।

RNN-এর সমস্যাসমূহ

- Vanishing / Exploding Gradient: বাক্য লম্বা হলে Backpropagation-এর সময় Gradient অনেক ছোট (vanish) অথবা অনেক বড় (explode) হয়ে যায়, ফলে মডেল দূরবর্তী শব্দের মধ্যে সম্পর্ক শিখতে পারে না।
- Long-Range Dependency-তে দুর্বলতা: বাক্যের শুরুর একটি শব্দ শেষের শব্দের সাথে সম্পর্কিত হলে, RNN সেই তথ্য একটিমাত্র Hidden State-এর মধ্যে বহন করতে গিয়ে হারিয়ে ফেলে।
- Parallelization-এর অভাব: h_t গণনা করতে h_{t-1} লাগবেই, তাই GPU-তে একসাথে (parallel) সব token প্রসেস করা যায় না — training ধীরগতির হয়।

২. LSTM (Long Short-Term Memory)

RNN-এর Vanishing Gradient সমস্যা সমাধানের জন্য LSTM তৈরি করা হয়েছিল। LSTM-এ একটি আলাদা Cell State (C_t) থাকে, যা তথ্যকে দীর্ঘ সময় ধরে অপরিবর্তিত রাখতে পারে, এবং তিনটি Gate দিয়ে নিয়ন্ত্রিত হয়:

- Forget Gate (f_t): পুরনো Cell State থেকে কতটুকু তথ্য ভুলে যেতে হবে তা নির্ধারণ করে।
- Input Gate (i_t): নতুন তথ্যের কতটুকু Cell State-এ যোগ হবে তা নির্ধারণ করে।
- Output Gate (o_t): Cell State থেকে কতটুকু তথ্য Hidden State হিসেবে বের হবে তা নির্ধারণ করে।

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

এখানে σ হলো Sigmoid function এবং $*$ হলো Element-wise গুণ (Hadamard Product)। Forget ও Input Gate-এর মাধ্যমে LSTM নির্বাচনীভাবে (selectively) তথ্য রাখা বা ফেলে দেওয়া শিখতে পারে, ফলে Long-Term Dependency অনেকটা ভালোভাবে ধরে রাখা যায়।

তবে LSTM-ও এখনও Sequential — C_t এবং h_t গণনা করতে C_{t-1} ও h_{t-1} প্রয়োজন। তাই Parallelization সম্ভব নয়, এবং লম্বা sequence-এর জন্য Training এখনও অনেক ধীর ও ব্যয়বহুল (computationally expensive)।

৩. Attention

RNN/LSTM পুরো পূর্ববর্তী sequence-কে একটিমাত্র Hidden State-এ সংকুচিত (compress) করতে বাধ্য করে — এটি একটি Information Bottleneck তৈরি করে। Attention Mechanism এই সমস্যার সমাধান করে এভাবে যে, মডেলকে পুরো sequence সংকুচিত না করে, প্রতিটি Output-এর জন্য সরাসরি Input-এর প্রাসঙ্গিক (relevant) অংশে ফোকাস করার সুযোগ দেয়।

Attention-এর মূল ধারণাটি তিনটি উপাদানের মাধ্যমে বোঝা যায়:

- Query (Q): "আমি কী খুঁজছি?" — বর্তমান token কী তথ্য জানতে চায়।
- Key (K): "আমার কাছে কী তথ্য আছে?" — প্রতিটি token নিজের সম্পর্কে যে পরিচয় (label) দেয়।
- Value (V): "সেই তথ্যের প্রকৃত বিষয়বস্তু কী?" — Query, Key-এর সাথে মিলে গেলে যে প্রকৃত তথ্য দেওয়া হয়।

সহজভাবে বলা যায়: Query, সব Key-এর সাথে তুলনা করে দেখে কোন token-গুলো সবচেয়ে বেশি প্রাসঙ্গিক, এবং সেই অনুযায়ী Value-গুলোর একটি Weighted Average নেওয়া হয়। এর সম্পূর্ণ গণিত STEP 4-এ বিস্তারিতভাবে দেখানো হয়েছে।

8. Transformer

২০১৭ সালে Vaswani et al. তাঁদের বিখ্যাত গবেষণাপত্র “Attention Is All You Need”-এ Transformer আর্কিটেকচার প্রকাশ করেন। এর মূল অবদান হলো — Transformer সম্পূর্ণভাবে Recurrence (RNN/LSTM-এর মতো ধাপে-ধাপে প্রসেসিং) বাদ দিয়ে শুধুমাত্র Attention Mechanism ব্যবহার করে পুরো sequence প্রসেস করে।

যেহেতু কোনো token-এর গণনা আগের token-এর উপর নির্ভরশীল নয়, তাই সব token-এর Attention একসাথে (Parallel-ভাবে) GPU-তে গণনা করা যায়। এর ফলে Training অনেক দ্রুত হয়, এবং Long-Range Dependency-ও অনেক ভালোভাবে ধরা পড়ে, কারণ যেকোনো দুটি token-এর মধ্যে দূরত্ব নির্বিশেষে সরাসরি Attention গণনা করা যায়।

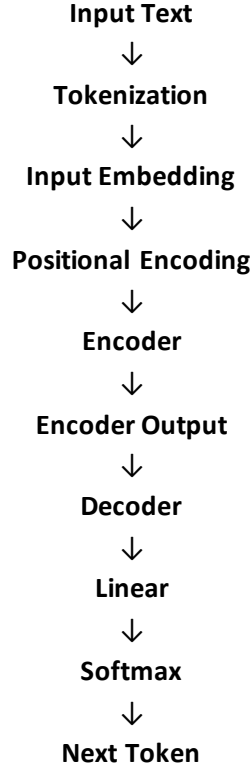
RNN / LSTM (Sequential) $\longrightarrow$ Transformer (Parallel)

এই কারণেই Transformer আজকের প্রায় সব বড় ভাষা মডেলের (Large Language Model) ভিত্তি — যেমন BERT, GPT, T5 ইত্যাদি সবই মূল Transformer আর্কিটেকচার থেকে উদ্ভূত।

অংশ ২: Transformer আর্কিটেকচার

মূল (Original) Transformer একটি Encoder-Decoder কাঠামো। Encoder ইনপুট বাক্যকে প্রসেস করে একটি Contextual Representation তৈরি করে, আর Decoder সেই Representation ব্যবহার করে ধাপে ধাপে (autoregressive) Output বাক্য তৈরি করে।

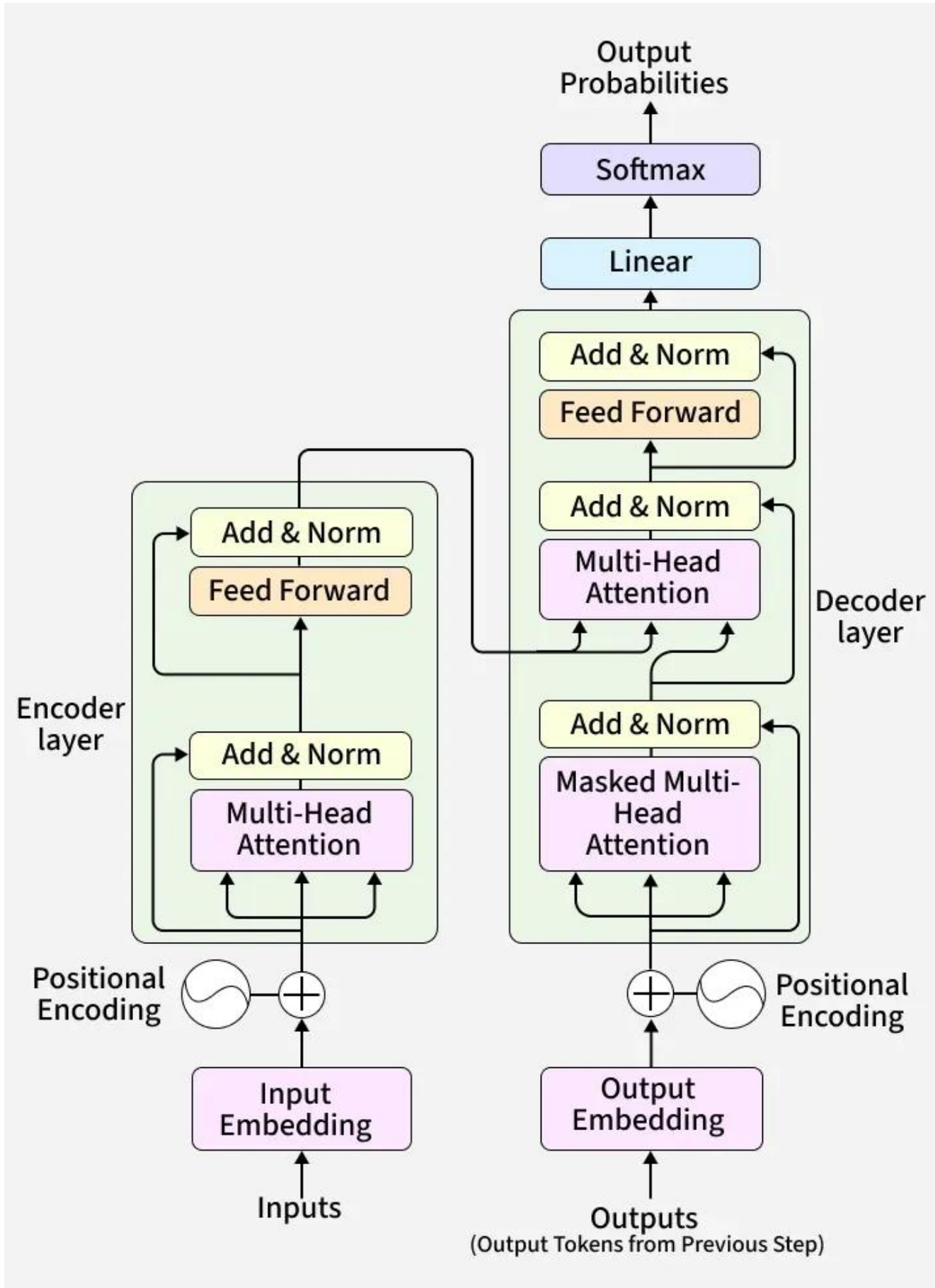
সামগ্রিক Data-Flow ডায়াগ্রাম



Encoder ও Decoder-এর ভেতরে তিন ধরনের Attention

- Encoder Self-Attention: Encoder-এর প্রতিটি token, একই sentence-এর সব token-এর (নিজেসহ) সাথে Attention গণনা করে। এখানে কোনো Masking নেই — প্রতিটি token সম্পূর্ণ বাক্যটি (আগে-পরে, সব দিকেই) দেখতে পারে।
- Decoder Masked Self-Attention: Decoder-এর প্রতিটি token শুধুমাত্র নিজের আগের token-গুলো দেখতে পারে, পরের token নয় — কারণ Inference-এর সময় ভবিষ্যতের token এখনো তৈরিই হয়নি। এটি একটি Causal Mask দিয়ে বাস্তবায়িত হয়।
- Decoder Cross-Attention: এখানে Query আসে Decoder থেকে, কিন্তু Key ও Value আসে Encoder-এর চূড়ান্ত Output থেকে। এভাবে Decoder প্রতিটি ধাপে Input বাক্যের প্রাসঙ্গিক অংশে ফোকাস করতে পারে।

Encoder-এর প্রতিটি Layer-এ থাকে: Multi-Head Self-Attention → Add & Norm → Feed Forward Network → Add & Norm। Decoder-এর প্রতিটি Layer-এ থাকে: Masked Multi-Head Self-Attention → Add & Norm → Cross-Attention → Add & Norm → Feed Forward Network → Add & Norm। এই নোটে আমরা একটি করে Encoder Layer ও Decoder Layer সম্পূর্ণভাবে গণনা করব।



অংশ ৩: একটি নির্দিষ্ট সংখ্যাগত উদাহরণ (One Consistent Numerical Example)

পুরো ডকুমেন্টে আমরা এই বাক্যটি ব্যবহার করব:

"I am trying to learn Machine Learning"

এই বাক্যে মোট ৭টি token আছে (৭ শব্দ, প্রতিটি একটি করে token হিসেবে ধরা হয়েছে)। ধরে নিচ্ছি Tokenizer এই ৭টি token-কে নিচের Token ID-গুলো দিয়েছে:

	Token ID
I	146
am	52
trying	894
to	12
learn	1205
Machine	3410
Learning	2180

আমরা মডেলের Embedding Dimension হিসেবে খুবই ছোট একটি মান বেছে নিচ্ছি, যাতে প্রতিটি ভেক্টর হাতে-কলমে গণনা করা যায়:

d_model = 4, Attention Head সংখ্যা h = 2, Sequence Length n = 7

এই ছোট dimension বাস্তব Transformer-এ ব্যবহৃত হয় না (বাস্তবে d_model সাধারণত 512, 768 বা তার বেশি হয়), কিন্তু এখানে শেখার উদ্দেশ্যে d_model = 4 বেছে নেওয়া হয়েছে যাতে প্রতিটি Matrix Multiplication হাতে যাচাই করা সম্ভব হয়।

অংশ ৪: Encoder — সম্পূর্ণ গাণিতিক হিসাব

STEP 1: Input

Purpose

মডেল সরাসরি Text বোঝে না — তাই Text-কে প্রথমে ছোট ছোট একক (token)-এ ভাগ করতে হয়, এবং প্রতিটি token-কে একটি সংখ্যায় (Token ID) রূপান্তর করতে হয়, যাতে তা গাণিতিকভাবে প্রসেস করা যায়।

What Actually Happens

আমাদের বাক্য "I am trying to learn Machine Learning" -কে Tokenizer ৭টি token-এ ভাগ করে এবং প্রতিটিকে Vocabulary-তে থাকা একটি অনন্য (unique) সংখ্যা (Token ID) বরাদ্দ করে।

Result

	Token ID
I	146
am	52
trying	894
to	12
learn	1205
Machine	3410
Learning	2180

Sequence Length $n = 7$ । এই ৭টি Token ID এখন পরের ধাপে (Embedding) ইনপুট হিসেবে ব্যবহৃত হবে।

STEP 2: Input Embedding

Purpose

Token ID নিজে একটি নিছক সংখ্যা (146, 52, ...) — এতে শব্দের অর্থ সম্পর্কে কোনো তথ্য নেই। Embedding প্রতিটি Token ID-কে একটি ঘন (dense) ভেক্টরে রূপান্তর করে, যেখানে কাছাকাছি অর্থের শব্দের ভেক্টর একে অপরের কাছাকাছি থাকে। এই ভেক্টরগুলোই Training-এর মাধ্যমে শেখা হয়।

What Actually Happens

প্রতিটি Token ID একটি বড় Embedding Matrix W_E থেকে তার নিজস্ব সারি (row) "lookup" করে নেয়। W_E -এর আকার (shape):

$$W_E \in \mathbb{R}^{V \times d_{\text{model}}}$$

এখানে V হলো Vocabulary-এর আকার (মোট কতগুলো ভিন্ন token আছে) এবং $d_{\text{model}} = 4$ । এই Lookup অপারেশনকে গাণিতিকভাবে One-Hot Encoding দিয়েও প্রকাশ করা যায়:

$$e_i = \text{OneHot}(t_i) W_E$$

অর্থাৎ token t_i -কে প্রথমে একটি One-Hot ভেক্টরে রূপান্তর করা হয় (যেখানে শুধু t_i -এর অবস্থানে 1, বাকি সব জায়গায় 0), এবং তারপর সেটিকে W_E দিয়ে গুণ করলে তা কার্যত W_E -এর সেই নির্দিষ্ট সারিটি "select" করে ফেলে — এটিই Lookup।

Full Numerical Calculation

নিচে প্রতিটি token-এর জন্য (illustrative, শেখানোর উদ্দেশ্যে বেছে নেওয়া) Embedding ভেক্টর দেখানো হলো — এগুলোই যেন W_E থেকে lookup করে পাওয়া গেছে:

	dim1	dim2	dim3	dim4
I	0.1	0.2	0.3	0.4
am	0.15	0.1	0.25	0.05
trying	0.05	0.3	0.1	0.2
to	0.2	0.05	0.15	0.1
learn	0.3	0.25	0.05	0.15
Machine	0.25	0.15	0.2	0.05
Learning	0.1	0.35	0.1	0.25

Result

$$E \in \mathbb{R}^{7 \times 4}$$

এই সম্পূর্ণ 7×4 ম্যাট্রিক্সটিই আমাদের Embedding Matrix E, যা এখন Positional Encoding-এর সাথে যোগ করা হবে।

STEP 3: Positional Encoding

Purpose

Transformer-এ কোনো Recurrence নেই, তাই মডেল নিজে থেকে জানে না কোন token বাক্যে কোন অবস্থানে (position) আছে — Attention Mechanism শুধু token-গুলোর মধ্যে সম্পর্ক দেখে, ক্রম (order) দেখে না। এই সমস্যার সমাধানে প্রতিটি token-এর Embedding-এর সাথে তার Position অনুযায়ী একটি নির্দিষ্ট ভেক্টর (Positional Encoding) যোগ করা হয়।

Mathematical Formula

মূল Transformer গবেষণাপত্রে ব্যবহৃত Sinusoidal Positional Encoding:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

এখানে pos হলো token-এর অবস্থান (0 থেকে শুরু), i হলো dimension-এর index (জোড়/বিজোড় জোড়া), এবং $d_{\text{model}} = 4$ । যেহেতু $d_{\text{model}} = 4$, তাই i মাত্র 0 এবং 1 হতে পারে — অর্থাৎ dimension 0,1 একটি sin-cos জোড়া ($2i = 0$) এবং dimension 2,3 আরেকটি sin-cos জোড়া ($2i = 2$)।

Full Numerical Calculation

Position 0 (token "I")-এর জন্য:

- dim 0: $\sin(0 / 10000^0) = \sin(0) = 0$
- dim 1: $\cos(0 / 10000^0) = \cos(0) = 1$
- dim 2: $\sin(0 / 10000^{0.5}) = \sin(0) = 0$
- dim 3: $\cos(0 / 10000^{0.5}) = \cos(0) = 1$

Position 1 (token "am")-এর জন্য:

- dim 0: $\sin(1 / 10000^0) = \sin(1) = 0.8415$
- dim 1: $\cos(1 / 10000^0) = \cos(1) = 0.5403$

- dim 2: $\sin(1 / 100) = \sin(0.01) = 0.01$
- dim 3: $\cos(1 / 100) = \cos(0.01) = 1$

একইভাবে (একই সূত্র প্রয়োগ করে, ভিন্ন pos বসিয়ে) বাকি সব 7টি position (pos = 0 থেকে 6) -এর জন্য গণনা করলে সম্পূর্ণ PE ম্যাট্রিক্স পাওয়া যায়:

	dim1 (sin)	dim2 (cos)	dim3 (sin)	dim4 (cos)
I	0	1	0	1
am	0.8415	0.5403	0.01	1
trying	0.9093	-0.4161	0.02	0.9998
to	0.1411	-0.99	0.03	0.9996
learn	-0.7568	-0.6536	0.04	0.9992
Machine	-0.9589	0.2837	0.05	0.9988
Learning	-0.2794	0.9602	0.06	0.9982

Result

$$PE \in \mathbb{R}^{7 \times 4}$$

এখন Token Embedding ও Positional Encoding যোগ করে চূড়ান্ত Input Representation X তৈরি হয়:

$$X = E + PE$$

প্রতিটি সারি আলাদাভাবে যোগ করে (যেমন "I": $[0.10, 0.20, 0.30, 0.40] + [0, 1, 0, 1] = [0.10, 1.20, 0.30, 1.40]$) সম্পূর্ণ X ম্যাট্রিক্স পাওয়া যায়:

	dim1	dim2	dim3	dim4
I	0.1	1.2	0.3	1.4
am	0.9915	0.6403	0.26	1.05
trying	0.9593	-0.1161	0.12	1.1998
to	0.3411	-0.94	0.18	1.0996
learn	-0.4568	-0.4036	0.09	1.1492
Machine	-0.7089	0.4337	0.25	1.0488
Learning	-0.1794	1.3102	0.16	1.2482

এই X (আকার 7×4) ম্যাট্রিক্সটিই এখন Encoder-এর Multi-Head Self-Attention Layer-এ প্রবেশ করবে।

STEP 4: Multi-Head Self-Attention

Purpose

এটি Transformer-এর সবচেয়ে গুরুত্বপূর্ণ অংশ। এই ধাপে প্রতিটি token অন্য সব token-এর সাথে নিজের সম্পর্ক (relevance) গণনা করে, এবং সেই সম্পর্ক অনুযায়ী অন্য token-গুলোর তথ্য নিজের মধ্যে যুক্ত করে নেয়। এভাবেই প্রতিটি token একটি Context-aware (প্রসঙ্গ-সচেতন) representation পায়।

What Actually Happens

প্রথমে x থেকে তিনটি নতুন ম্যাট্রিক্স তৈরি করা হয় — Query (Q), Key (K), এবং Value (V) — পৃথক তিনটি Weight Matrix দিয়ে গুণ করে:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

W_Q , W_K , W_V -এর আকার প্রতিটি 4×4 (illustrative, ছোট সংখ্যা দিয়ে সংজ্ঞায়িত):

	c1	c2	c3	c4
row1	0.1	0.2	0	0.1
row2	0.3	0.1	0.2	0
row3	0	0.2	0.1	0.3
row4	0.2	0	0.3	0.1

W_Q ম্যাট্রিক্স (উপরে)। একইভাবে W_K এবং W_V :

	c1	c2	c3	c4
row1	0.2	0	0.1	0.2
row2	0.1	0.3	0	0.1
row3	0	0.1	0.2	0
row4	0.3	0.2	0	0.2

W_K ম্যাট্রিক্স (উপরে)।

	c1	c2	c3	c4
row1	0.1	0.1	0.2	0
row2	0	0.2	0.1	0.1
row3	0.2	0	0.1	0.2
row4	0.1	0.1	0	0.2

W_V ম্যাট্রিক্স (উপরে)।

Full Numerical Calculation — Q, K, V

x (7×4) কে W_Q (4×4) দিয়ে গুণ করে Q (7×4) পাওয়া যায় — যেমন "I" token-এর Q ভেক্টর গণনা: x -এর প্রথম সারি $[0.10, 1.20, 0.30, 1.40]$ -কে W_Q -এর প্রতিটি কলামের সাথে Dot Product নিলে Q -এর প্রথম সারি পাওয়া যায়। একইভাবে বাকি ৬টি সারির জন্যও গণনা করে সম্পূর্ণ Q ম্যাট্রিক্স তৈরি হয়:

	q1	q2	q3	q4
I	0.65	0.2	0.69	0.24
am	0.5012	0.3143	0.469	0.2821
trying	0.301	0.2042	0.3487	0.2519
to	-0.028	0.0102	0.1599	0.1981
learn	0.0631	-0.1137	0.273	0.0962

	q1	q2	q3	q4
Machine	0.269	-0.0484	0.4264	0.109
Learning	0.6247	0.1271	0.6525	0.1549

সম্পূর্ণ K ম্যাট্রিক্স ($X \times W_K$):

	k1	k2	k3	k4
I	0.56	0.67	0.07	0.42
am	0.5773	0.4281	0.1511	0.4723
trying	0.5402	0.2171	0.1199	0.4202
to	0.3041	-0.0441	0.0701	0.1941
learn	0.213	0.1177	-0.0277	0.0981
Machine	0.2162	0.3648	-0.0209	0.1113
Learning	0.4696	0.6587	0.0141	0.3448

সম্পূর্ণ V ম্যাট্রিক্স ($X \times W_V$):

	v1	v2	v3	v4
I	0.21	0.39	0.17	0.46
am	0.2561	0.3322	0.2883	0.326
trying	0.2399	0.1927	0.1922	0.2523
to	0.1801	-0.0439	-0.0078	0.1619
learn	0.0872	-0.0115	-0.1227	0.2075
Machine	0.084	0.1207	-0.0734	0.3031
Learning	0.1389	0.3689	0.1111	0.4126

এই ধাপ থেকে multi-head-এ ভাগ করা

এখানে $h = 2$ (দুইটি Attention Head) এবং প্রতিটি Head-এর dimension $d_k = 4/2 = 2$ । বাস্তব বাস্তবায়নে (এবং এই নোটেও) Q, K, V-এর 4টি column-কে দুই ভাগে ভাগ করা হয় — প্রথম ২টি column Head 1-এর জন্য, শেষ ২টি column Head 2-এর জন্য। এতে আলাদা করে দ্বিতীয় সেট Weight Matrix সংজ্ঞায়িত করার দরকার হয় না, কিন্তু গাণিতিকভাবে এটি প্রতিটি Head-কে independent Q, K, V দেওয়ার সমতুল্য।

প্রথমে একটি একক (Single-Head, Full $d_k = 4$) Attention হিসাব — ধারণা স্পষ্ট করার জন্য

Attention-এর মূল সূত্র (Scaled Dot-Product Attention):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Attention Score (QK^T)

$$\text{Score} = QK^T$$

Q (7×4) কে K^T (4×7) দিয়ে গুণ করলে একটি 7×7 ম্যাট্রিক্স পাওয়া যায়, যেখানে (i, j) উপাদানটি বোঝায় token i , token j -কে কতটা গুরুত্ব দিচ্ছে (raw score, এখনও normalize করা হয়নি)। সম্পূর্ণ 7×7 Score ম্যাট্রিক্স:

	I	am	trying	to	learn	Machine	Learning
I	0.6471	0.6785	0.5781	0.2838	0.1665	0.2258	0.5294
am	0.6426	0.6281	0.5138	0.2262	0.1585	0.2447	0.5463
trying	0.4356	0.4329	0.3546	0.1559	0.1032	0.1604	0.3677
to	0.0856	0.1059	0.0895	0.0407	0.0103	0.0164	0.0641
learn	0.0187	0.0744	0.0826	0.062	0.0019	-0.0228	-0.0083
Machine	0.1938	0.2505	0.2317	0.135	0.0505	0.0437	0.138
Learning	0.5458	0.5869	0.5084	0.2602	0.1452	0.1851	0.4397

Scaling

$$\text{Scaled Score} = \frac{QK^T}{\sqrt{d_k}}$$

$d_k = 4$ হওয়ায় $\sqrt{d_k} = 2$ । Score-এর প্রতিটি উপাদানকে 2 দিয়ে ভাগ করলে scaled score পাওয়া যায়। Scaling না করলে dimension বড় হলে dot-product-এর মান অনেক বড় হয়ে যেত, যার ফলে Softmax-এর gradient প্রায় শূন্য (vanishing gradient) হয়ে যেত — তাই স্থিতিশীল (stable) gradient-এর জন্য $\sqrt{d_k}$ দিয়ে ভাগ করা প্রয়োজন। সম্পূর্ণ Scaled Score ম্যাট্রিক্স:

	I	am	trying	to	learn	Machine	Learning
I	0.3236	0.3393	0.2891	0.1419	0.0832	0.1129	0.2647
am	0.3213	0.314	0.2569	0.1131	0.0792	0.1223	0.2731
trying	0.2178	0.2165	0.1773	0.0779	0.0516	0.0802	0.1838
to	0.0428	0.053	0.0448	0.0204	0.0051	0.0082	0.0321
learn	0.0093	0.0372	0.0413	0.031	0.001	-0.0114	-0.0041
Machine	0.0969	0.1252	0.1158	0.0675	0.0252	0.0219	0.069
Learning	0.2729	0.2934	0.2542	0.1301	0.0726	0.0925	0.2198

Softmax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

প্রতিটি সারির জন্য আলাদাভাবে Softmax প্রয়োগ করা হয়, যাতে প্রতিটি সারির যোগফল 1 হয় (এগুলোই Attention Probability)। উদাহরণস্বরূপ, "I" token-এর সারি (Scaled Score):

[0.3236, 0.3393, 0.2891, 0.1419, 0.0832, 0.1129, 0.2647] — এই ৭টি মানের প্রতিটির উপর e^z প্রয়োগ করে (exponential নেওয়া হয়), তারপর সবগুলোর যোগফল দিয়ে ভাগ করা হয়:

e^z মানসমূহ: [1.3821, 1.4040, 1.3352, 1.1525, 1.0868, 1.1195, 1.3030], যোগফল $\sum e^z = 8.7831$

প্রতিটিকে যোগফল দিয়ে ভাগ করলে: [0.1574, 0.1598, 0.1520, 0.1312, 0.1237, 0.1275, 0.1484] (যোগফল ≈ 1.0000 — যাচাই করা গেল)

একইভাবে বাকি ৬টি সারির (token) জন্যও exponential ও normalization প্রয়োগ করে সম্পূর্ণ Attention Probability ম্যাট্রিক্স (7×7) পাওয়া যায়, যার প্রতিটি সারির যোগফল 1:

	I	am	trying	to	learn	Machine	Learning
I	0.1574	0.1598	0.152	0.1312	0.1237	0.1275	0.1484
am	0.1587	0.1576	0.1488	0.1289	0.1246	0.1301	0.1513
trying	0.1535	0.1533	0.1474	0.1335	0.13	0.1338	0.1484
to	0.1448	0.1462	0.145	0.1415	0.1394	0.1398	0.1432
learn	0.142	0.1461	0.1466	0.1451	0.1409	0.1391	0.1401
Machine	0.146	0.1502	0.1488	0.1418	0.1359	0.1354	0.142
Learning	0.1545	0.1577	0.1517	0.134	0.1265	0.129	0.1466

Weighted Values

Attention Probability ম্যাট্রিক্সকে V (7×4) দিয়ে গুণ করলে চূড়ান্ত Attention Output পাওয়া যায় — প্রতিটি token এখন অন্য সব token-এর Value-এর একটি ওজনযুক্ত (weighted) গড়:

	a1	a2	a3	a4
I	0.1762	0.2067	0.093	0.3096
am	0.1754	0.2073	0.092	0.3105
trying	0.1741	0.2028	0.0883	0.3082

	a1	a2	a3	a4
to	0.1719	0.1949	0.0821	0.3042
learn	0.172	0.1927	0.0814	0.3027
Machine	0.1733	0.1964	0.0848	0.3044
Learning	0.1756	0.2042	0.0912	0.3083

এটি একক-হেড (single-head, $d_k=4$) Attention-এর সম্পূর্ণ ফলাফল $A \in \mathbb{R}^{7 \times 4}$ । এখন আমরা দেখব কীভাবে Multi-Head Attention-এ এই একই গণনা দুইটি ছোট Head-এ ভাগ করে করা হয়।

Multi-Head Attention (h = 2)

একটিমাত্র Attention-এর বদলে একাধিক "Head" ব্যবহার করলে মডেল একই সাথে বিভিন্ন ধরনের সম্পর্ক (যেমন এক Head Grammar-ভিত্তিক সম্পর্ক শিখতে পারে, আরেক Head Semantic/অর্থগত সম্পর্ক শিখতে পারে) শিখতে পারে। প্রতিটি Head স্বাধীনভাবে ছোট dimension-এ ($d_k = 2$) Attention গণনা করে, এবং শেষে সব Head-এর ফলাফল একত্র (concatenate) করা হয়।

$$head_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$

Head 1 ব্যবহার করে Q, K, V-এর প্রথম ২টি column (dim 1,2):

	q1	q2
I	0.65	0.2
am	0.5012	0.3143
trying	0.301	0.2042
to	-0.028	0.0102
learn	0.0631	-0.1137
Machine	0.269	-0.0484
Learning	0.6247	0.1271

(এটি Q_1 — Head 1-এর Query। একইভাবে K_1 এবং V_1 পাওয়া যায় K ও V-এর প্রথম দুই column থেকে।)

	k1	k2
I	0.56	0.67
am	0.5773	0.4281
trying	0.5402	0.2171
to	0.3041	-0.0441
learn	0.213	0.1177
Machine	0.2162	0.3648
Learning	0.4696	0.6587
	v1	v2
I	0.21	0.39
am	0.2561	0.3322
trying	0.2399	0.1927
to	0.1801	-0.0439
learn	0.0872	-0.0115
Machine	0.084	0.1207
Learning	0.1389	0.3689

Head 1 — Score, Scale, Softmax, Output

d_k (per head) = 2, তাই $\sqrt{d_k} = 1.4142$ । $Q_1 K_1^T / \sqrt{2}$ হিসাব করে সম্পূর্ণ 7×7 Scaled Score:

	I	am	trying	to	learn	Machine	Learning
I	0.3521	0.3259	0.279	0.1335	0.1146	0.151	0.309
am	0.3474	0.2998	0.2397	0.098	0.1017	0.1577	0.3128
trying	0.216	0.1847	0.1463	0.0584	0.0624	0.0987	0.1951
to	-0.0062	-0.0083	-0.0091	-0.0063	-0.0034	-0.0016	-0.0045
learn	-0.0289	-0.0087	0.0066	0.0171	0	-0.0197	-0.032
Machine	0.0836	0.0951	0.0953	0.0593	0.0365	0.0286	0.0668
Learning	0.3076	0.2935	0.2582	0.1304	0.1047	0.1283	0.2667

এর উপর সারি-ভিত্তিক Softmax প্রয়োগ করলে Head 1-এর Attention Probability ম্যাট্রিক্স:

	I	am	trying	to	learn	Machine	Learning
I	0.1595	0.1553	0.1482	0.1281	0.1257	0.1304	0.1527
am	0.1611	0.1536	0.1447	0.1256	0.126	0.1333	0.1557

	I	am	trying	to	learn	Machine	Learning
trying	0.1543	0.1495	0.1439	0.1318	0.1323	0.1372	0.1511
to	0.1428	0.1425	0.1424	0.1428	0.1432	0.1434	0.143
learn	0.1401	0.1429	0.1451	0.1467	0.1442	0.1414	0.1396
Machine	0.1453	0.147	0.147	0.1418	0.1386	0.1375	0.1429
Learning	0.1566	0.1544	0.149	0.1311	0.1278	0.1309	0.1503

এই Probability ম্যাট্রিককে V_1 দিয়ে গুণ করলে $head_1$ (7×2) পাওয়া যায়:

	h1_1	h1_2
I	0.175	0.2074
am	0.1743	0.2083
trying	0.173	0.2025
to	0.1707	0.1926
learn	0.1711	0.1906
Machine	0.1724	0.1953
Learning	0.1748	0.2051

Head 2 — Score, Scale, Softmax, Output

একই প্রক্রিয়া, কিন্তু Q, K, V-এর শেষ ২টি column (dim 3,4) ব্যবহার করে (Q_2, K_2, V_2):

	q3	q4
I	0.69	0.24
am	0.469	0.2821
trying	0.3487	0.2519
to	0.1599	0.1981
learn	0.273	0.0962
Machine	0.4264	0.109
Learning	0.6525	0.1549
	k3	k4
I	0.07	0.42
am	0.1511	0.4723
trying	0.1199	0.4202
to	0.0701	0.1941
learn	-0.0277	0.0981
Machine	-0.0209	0.1113
Learning	0.0141	0.3448
	v3	v4
I	0.17	0.46
am	0.2883	0.326
trying	0.1922	0.2523
to	-0.0078	0.1619
learn	-0.1227	0.2075
Machine	-0.0734	0.3031
Learning	0.1111	0.4126

Scaled Score ($Q_2 K_2^T / V_2$):

	I	am	trying	to	learn	Machine	Learning
I	0.1054	0.1539	0.1298	0.0672	0.0031	0.0087	0.0654
am	0.107	0.1444	0.1236	0.062	0.0104	0.0153	0.0734
trying	0.0921	0.1214	0.1044	0.0519	0.0107	0.0147	0.0649
to	0.0667	0.0832	0.0724	0.0351	0.0106	0.0132	0.0499
learn	0.0421	0.0613	0.0517	0.0267	0.0013	0.0035	0.0262
Machine	0.0535	0.082	0.0685	0.0361	-0.0008	0.0023	0.0308

	I	am	trying	to	learn	Machine	Learning
Learning	0.0783	0.1215	0.1013	0.0536	-0.002	0.0026	0.0442

Softmax-এর পর Attention Probability:

	I	am	trying	to	learn	Machine	Learning
I	0.1469	0.1542	0.1505	0.1414	0.1326	0.1333	0.1411
am	0.1471	0.1527	0.1496	0.1406	0.1336	0.1342	0.1422
trying	0.1466	0.1509	0.1484	0.1408	0.1351	0.1356	0.1426
to	0.1456	0.148	0.1464	0.1411	0.1377	0.138	0.1432
learn	0.1445	0.1473	0.1459	0.1423	0.1387	0.139	0.1422
Machine	0.1449	0.1491	0.1471	0.1424	0.1372	0.1377	0.1416
Learning	0.1458	0.1522	0.1492	0.1422	0.1345	0.1352	0.1409

Probability $\times V_2 = \text{head}_2 (7 \times 2)$:

	h2_1	h2_2
I	0.0869	0.3049
am	0.0863	0.305
trying	0.0852	0.3049
to	0.0834	0.3045
learn	0.0826	0.304
Machine	0.0836	0.3041
Learning	0.0855	0.3044

Concatenation এবং Output Projection

$$\text{Concat}(\text{head}_1, \text{head}_2)$$

$\text{head}_1 (7 \times 2)$ এবং $\text{head}_2 (7 \times 2)$ -কে পাশাপাশি জোড়া দিলে (concatenate) 7×4 আকারের একটি ম্যাট্রিক্স পাওয়া যায়:

	h1_1	h1_2	h2_1	h2_2
I	0.175	0.2074	0.0869	0.3049
am	0.1743	0.2083	0.0863	0.305
trying	0.173	0.2025	0.0852	0.3049
to	0.1707	0.1926	0.0834	0.3045
learn	0.1711	0.1906	0.0826	0.304
Machine	0.1724	0.1953	0.0836	0.3041
Learning	0.1748	0.2051	0.0855	0.3044

এরপর এই Concatenated ম্যাট্রিক্সকে একটি Output Projection Matrix $W_O (4 \times 4)$ দিয়ে গুণ করা হয়:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

W_O ম্যাট্রিক্স:

	c1	c2	c3	c4
row1	0.2	0.1	0	0.2
row2	0.1	0.2	0.1	0
row3	0	0.1	0.2	0.1
row4	0.2	0	0.1	0.2

Result

$\text{Concat}(\text{head}_1, \text{head}_2) \times W_O = \text{MultiHeadOutput}$ — এটিই Multi-Head Self-Attention-এর চূড়ান্ত ফলাফল (7×4):

	dim1	dim2	dim3	dim4
I	0.1167	0.0677	0.0686	0.1047
am	0.1167	0.0677	0.0686	0.1045
trying	0.1158	0.0663	0.0678	0.1041
to	0.1143	0.0639	0.0664	0.1034
learn	0.1141	0.0635	0.066	0.1033
Machine	0.1148	0.0647	0.0667	0.1037
Learning	0.1163	0.067	0.068	0.1044

STEP 5: Add & Norm (প্রথম)

Residual Connection-এর Purpose

Attention Layer অনেক গভীর নেটওয়ার্কের একটি অংশ। মূল Input X-কে সরাসরি Attention-এর Output-এর সাথে যোগ করে দিলে (Residual/Skip Connection), Gradient সহজে পেছনের দিকে (backward) প্রবাহিত হতে পারে এবং মূল তথ্য (original information) হারিয়ে যায় না।

$$R = X + \text{MultiHeadOutput}$$

X এবং MultiHeadOutput — উভয়েই 7x4 — সরাসরি element-wise যোগ করলে R পাওয়া যায়:

	dim1	dim2	dim3	dim4
I	0.2167	1.2677	0.3686	1.5047
am	1.1082	0.708	0.3286	1.1544
trying	1.0751	-0.0498	0.1878	1.3039
to	0.4554	-0.8761	0.2464	1.2029
learn	-0.3427	-0.3402	0.156	1.2525
Machine	-0.5941	0.4983	0.3166	1.1524
Learning	-0.0631	1.3772	0.228	1.3526

Layer Normalization-এর Purpose

Layer-এর গভীরতা বাড়ার সাথে সাথে Activation-এর মান খুব বড় বা খুব ছোট হয়ে যেতে পারে, যা Training-কে অস্থির করে তোলে। Layer Normalization প্রতিটি token-ভেক্টরকে (row-wise, dimension জুড়ে) normalize করে — অর্থাৎ প্রতিটি ভেক্টরের নিজস্ব Mean ও Variance ব্যবহার করে তাকে স্থিতিশীল একটি স্কেলে আনে।

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

$$\text{LayerNorm}(x_i) = \gamma \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

এই নোটে $\gamma = 1$ এবং $\beta = 0$ ধরা হয়েছে (অর্থাৎ কোনো অতিরিক্ত scale/shift নেই, শুধু normalization)।

Full Numerical Calculation — সাতটি token-এর জন্যই

- I: $R = [0.2167, 1.2677, 0.3686, 1.5047] \rightarrow \mu = 0.8394, \sigma^2 = 0.3088$
- am: $R = [1.1082, 0.7080, 0.3286, 1.1544] \rightarrow \mu = 0.8248, \sigma^2 = 0.1122$
- trying: $R = [1.0751, -0.0498, 0.1878, 1.3039] \rightarrow \mu = 0.6292, \sigma^2 = 0.3275$
- to: $R = [0.4554, -0.8761, 0.2464, 1.2029] \rightarrow \mu = 0.2572, \sigma^2 = 0.5545$
- learn: $R = [-0.3427, -0.3402, 0.1560, 1.2525] \rightarrow \mu = 0.1814, \sigma^2 = 0.4236$
- Machine: $R = [-0.5941, 0.4983, 0.3166, 1.1524] \rightarrow \mu = 0.3433, \sigma^2 = 0.3895$
- Learning: $R = [-0.0631, 1.3772, 0.2280, 1.3526] \rightarrow \mu = 0.7237, \sigma^2 = 0.4218$

প্রতিটি dimension x_i থেকে μ বিয়োগ করে $\sqrt{\sigma^2 + \epsilon}$ দিয়ে ভাগ করলে ($\gamma=1, \beta=0$) normalize করা ভেক্টর পাওয়া যায়। সম্পূর্ণ LayerNorm আউটপুট (7x4):

	dim1	dim2	dim3	dim4
I	-1.1205	0.7706	-0.8472	1.197
am	0.8459	-0.3486	-1.4813	0.984

	dim1	dim2	dim3	dim4
trying	0.7791	-1.1866	-0.7714	1.1789
to	0.2662	-1.5218	-0.0145	1.27
learn	-0.8052	-0.8013	-0.0391	1.6456
Machine	-1.502	0.2484	-0.0428	1.2964
Learning	-1.2113	1.0062	-0.7632	0.9683

Result

এই LN1 ম্যাট্রিক্সই এখন Feed Forward Network-এ প্রবেশ করবে।

STEP 6: Feed Forward Network (FFN)

Purpose

Attention শুধু token-গুলোর মধ্যে তথ্য আদান-প্রদান (mixing) করে — এটি একটি Linear (রৈখিক) অপারেশন। FFN প্রতিটি token-এর representation-এ একটি Non-linear রূপান্তর যোগ করে, যা মডেলকে আরও জটিল (complex) প্যাটার্ন শিখতে সাহায্য করে। এটি প্রতিটি token-এর জন্য আলাদাভাবে (position-wise), কিন্তু একই Weight দিয়ে প্রয়োগ করা হয়।

Mathematical Formula

$$FFN(x) = W_2 \text{ReLU}(W_1x + b_1) + b_2$$

W_1 (4×4), b_1 , W_2 (4×4), b_2 — এই ধাপের Weight ও Bias:

	c1	c2	c3	c4
row1	0.2	0.1	0	0.3
row2	0.1	0.2	0.3	0
row3	0	0.3	0.2	0.1
row4	0.3	0	0.1	0.2

$b_1 = [0.1000, 0.0000, 0.1000, 0.0000]$

	c1	c2	c3	c4
row1	0.1	0.2	0	0.1
row2	0.2	0	0.1	0.2
row3	0	0.1	0.2	0
row4	0.1	0	0.2	0.1

$b_2 = [0.0000, 0.1000, 0.0000, 0.1000]$

Full Numerical Calculation — সব ৭টি token-এর জন্য

$$z = W_1x + b_1$$

প্রতিটি token vector x (LN1-এর একটি সারি)-কে W_1 দিয়ে গুণ করে b_1 যোগ করলে z পাওয়া যায়। সম্পূর্ণ z_1 (7×4):

	z1	z2	z3	z4
I	0.3121	-0.2121	0.2814	-0.1814
am	0.5295	-0.4295	-0.2024	0.3024
trying	0.4908	-0.3908	-0.2924	0.3924
to	0.3821	-0.2821	-0.2324	0.3324
learn	0.3525	-0.2525	0.0164	0.0836
Machine	0.2134	-0.1134	0.2956	-0.1956
Learning	0.2488	-0.1488	0.3461	-0.2461

$$a = \text{ReLU}(z) = \max(0, z)$$

ঋণাত্মক (negative) মানগুলোকে 0 করে দিলে (ReLU) সম্পূর্ণ Activation ম্যাট্রিক্স:

	a1	a2	a3	a4
I	0.3121	0	0.2814	0
am	0.5295	0	0	0.3024
trying	0.4908	0	0	0.3924
to	0.3821	0	0	0.3324
learn	0.3525	0	0.0164	0.0836

	a1	a2	a3	a4
Machine	0.2134	0	0.2956	0
Learning	0.2488	0	0.3461	0

$$FFN(x) = W_2a + b_2$$

Activation-কে W_2 দিয়ে গুণ করে b_2 যোগ করলে চূড়ান্ত FFN Output (7×4) পাওয়া যায়:

	f1	f2	f3	f4
I	0.0312	0.1906	0.0563	0.1312
am	0.0832	0.2059	0.0605	0.1832
trying	0.0883	0.1982	0.0785	0.1883
to	0.0715	0.1764	0.0665	0.1715
learn	0.0436	0.1721	0.02	0.1436
Machine	0.0213	0.1722	0.0591	0.1213
Learning	0.0249	0.1844	0.0692	0.1249

STEP 7: Add & Norm (দ্বিতীয়)

আবার একটি Residual Connection: FFN-এর আগের Representation (LN1) এবং FFN-এর Output যোগ করা হয়:

$$R = \text{Previous Representation} + FFN(\text{Output})$$

	dim1	dim2	dim3	dim4
I	-1.0893	0.9611	-0.7909	1.3283
am	0.9291	-0.1427	-1.4208	1.1672
trying	0.8675	-0.9884	-0.6929	1.3672
to	0.3377	-1.3454	0.052	1.4415
learn	-0.7616	-0.6291	-0.0191	1.7892
Machine	-1.4806	0.4206	0.0164	1.4177
Learning	-1.1865	1.1906	-0.694	1.0932

এর উপর আবার Layer Normalization প্রয়োগ করা হয় (একই সূত্র, $\gamma=1$, $\beta=0$):

- I: $\mu = 0.1023$, $\sigma^2 = 1.1145$
- am: $\mu = 0.1332$, $\sigma^2 = 1.0484$
- trying: $\mu = 0.1383$, $\sigma^2 = 1.0006$
- to: $\mu = 0.1215$, $\sigma^2 = 0.9864$
- learn: $\mu = 0.0948$, $\sigma^2 = 1.0354$
- Machine: $\mu = 0.0935$, $\sigma^2 = 1.0861$
- Learning: $\mu = 0.1008$, $\sigma^2 = 1.1153$

চূড়ান্ত LayerNorm আউটপুট (7×4):

	dim1	dim2	dim3	dim4
I	-1.1287	0.8135	-0.8461	1.1612
am	0.7773	-0.2695	-1.5177	1.0099
trying	0.7289	-1.1264	-0.831	1.2285
to	0.2177	-1.4769	-0.0699	1.3291
learn	-0.8417	-0.7115	-0.1119	1.6652
Machine	-1.5105	0.3138	-0.074	1.2706
Learning	-1.2189	1.0319	-0.7526	0.9397

STEP 8: Encoder Output

এটিই একটি Encoder Layer-এর চূড়ান্ত আউটপুট — H_{enc} (7×4)। এর প্রতিটি সারি এখন সংশ্লিষ্ট token-এর একটি Contextual Representation, যেখানে বাক্যের অন্য সব token-এর তথ্য মিশে আছে।

	dim1	dim2	dim3	dim4
I	-1.1287	0.8135	-0.8461	1.1612
am	0.7773	-0.2695	-1.5177	1.0099
trying	0.7289	-1.1264	-0.831	1.2285
to	0.2177	-1.4769	-0.0699	1.3291
learn	-0.8417	-0.7115	-0.1119	1.6652
Machine	-1.5105	0.3138	-0.074	1.2706
Learning	-1.2189	1.0319	-0.7526	0.9397

এই H_{enc} ম্যাট্রিক্স পরবর্তীতে Decoder-এর Cross-Attention ধাপে Key (K) এবং Value (V) হিসেবে ব্যবহৃত হবে। (বাস্তব Transformer-এ একাধিক Encoder Layer পরপর বসানো হয় — $N \times \text{Encoder Layer}$ — কিন্তু এই নোটে একটি Layer সম্পূর্ণ দেখানো হয়েছে, বাকিগুলো একই প্রক্রিয়ার পুনরাবৃত্তি মাত্র, শুধু ভিন্ন Weight দিয়ে।)

অংশ ৫: Decoder — সম্পূর্ণ গাণিতিক হিসাব

Decoder-এর কাজ হলো Encoder-এর তৈরি করা Representation (H_{enc}) ব্যবহার করে, ধাপে ধাপে (একটির পর একটি token) Output বাক্য তৈরি করা। একে বলা হয় Autoregressive Generation — প্রতিটি নতুন token তৈরি হয় আগের সব token-এর উপর ভিত্তি করে।

STEP 9: Shifted-Right Input

Training-এর সময় Decoder-কে সম্পূর্ণ Target বাক্য একবারে দেখানো হয়, কিন্তু একটি বিশেষ কৌশলে — Target-কে একধাপ ডানদিকে সরিয়ে (shift right) এবং শুরুতে একটি বিশেষ <BOS> (Beginning of Sentence) token বসিয়ে Decoder-এর Input বানানো হয়। এর কারণ হলো, t -তম ধাপে Decoder-কে t -তম Target token অনুমান করতে হবে, কিন্তু ইনপুট হিসেবে শুধু $(t-1)$ -তম পর্যন্ত token দেখানো উচিত — যাতে মডেল ভবিষ্যতের উত্তর "দেখে না ফেলে"।

Target (যা Decoder-এর অনুমান করা উচিত):

I → am → trying → to → learn → Machine → Learning → <EOS>

Decoder Input (Shifted Right, শুরুতে <BOS>):

<BOS> → I → am → trying → to → learn → Machine → Learning

অর্থাৎ Decoder যখন <BOS> দেখে, তার Target হলো "I"; যখন <BOS>, "I" দেখে, তার Target হলো "am"; এভাবে শেষ পর্যন্ত "Learning" দেখলে তার Target হলো <EOS> (End of Sentence)।

STEP 10: Output (Decoder) Embedding

Encoder-এর মতোই, প্রতিটি Decoder Input token-কে একটি Embedding ভেক্টরে রূপান্তর করা হয়। যেহেতু "I", "am", "trying" ইত্যাদি token Encoder-এও ব্যবহৃত হয়েছিল, আমরা একই Embedding মান ব্যবহার করছি; শুধু নতুন বিশেষ token <BOS>-এর জন্য একটি নতুন (illustrative) ভেক্টর সংজ্ঞায়িত করা হলো:

<BOS> এর Embedding = [0.0500, 0.0500, 0.0500, 0.0500]

সম্পূর্ণ Decoder Embedding ম্যাট্রিক্স E_{dec} (8×4), ৮টি Decoder-Input token-এর জন্য:

	dim1	dim2	dim3	dim4
<BOS>	0.05	0.05	0.05	0.05
I	0.1	0.2	0.3	0.4
am	0.15	0.1	0.25	0.05
trying	0.05	0.3	0.1	0.2
to	0.2	0.05	0.15	0.1
learn	0.3	0.25	0.05	0.15
Machine	0.25	0.15	0.2	0.05
Learning	0.1	0.35	0.1	0.25

STEP 11: Positional Encoding (Decoder)

একই Sinusoidal সূত্র ব্যবহার করে, কিন্তু এবার ৪টি position (0 থেকে 7)-এর জন্য Positional Encoding গণনা করা হয়:

	dim1	dim2	dim3	dim4
<BOS>	0	1	0	1
I	0.8415	0.5403	0.01	1
am	0.9093	-0.4161	0.02	0.9998

	dim1	dim2	dim3	dim4
trying	0.1411	-0.99	0.03	0.9996
to	-0.7568	-0.6536	0.04	0.9992
learn	-0.9589	0.2837	0.05	0.9988
Machine	-0.2794	0.9602	0.06	0.9982
Learning	0.657	0.7539	0.0699	0.9976

$$X = E + PE$$

(এখানে $X = X_{dec}$, $E = E_{dec}$)। যোগ করার পর সম্পূর্ণ X_{dec} (8×4):

	dim1	dim2	dim3	dim4
<BOS>	0.05	1.05	0.05	1.05
I	0.9415	0.7403	0.31	1.4
am	1.0593	-0.3161	0.27	1.0498
trying	0.1911	-0.69	0.13	1.1996
to	-0.5568	-0.6036	0.19	1.0992
learn	-0.6589	0.5337	0.1	1.1488
Machine	-0.0294	1.1102	0.26	1.0482
Learning	0.757	1.1039	0.1699	1.2476

STEP 12: Masked Multi-Head Self-Attention

Purpose

Decoder Self-Attention-এ প্রতিটি token শুধুমাত্র নিজের এবং তার আগের token-গুলোর সাথে Attention করতে পারবে — পরের (ভবিষ্যতের) token দেখতে পারবে না। এটি নিশ্চিত করে যে Training এবং Inference-এর আচরণ একরকম থাকে (Inference-এর সময় ভবিষ্যতের token আদৌ পাওয়াই যায় না)।

Causal Mask

$$M = \begin{bmatrix} 0 & -\infty & -\infty & \dots \\ 0 & 0 & -\infty & \dots \\ 0 & 0 & 0 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

এখানে যে অবস্থানগুলোতে $(i, j) \ j > i$ (অর্থাৎ ভবিষ্যতের token), সেখানে $-\infty$ বসানো হয়; বাকি সব জায়গায় 0।

Q, K, V (Decoder Self-Attention)

X_{dec} -কে নতুন Weight Matrix W_Q , W_K , W_V (Decoder self-attention-এর নিজস্ব, Encoder-এর থেকে ভিন্ন) দিয়ে গুণ করে Q, K, V (প্রতিটি 8×4) পাওয়া যায়:

	q1	q2	q3	q4
<BOS>	0.32	0.115	0.225	0.32
I	0.3822	0.136	0.4993	0.3822
am	0.1477	0.0224	0.4488	0.1477
trying	0.0011	-0.043	0.2911	0.0011
to	-0.0665	-0.0224	0.1275	-0.0665
learn	0.1557	0.0734	0.108	0.1557
Machine	0.3239	0.163	0.2298	0.3239
Learning	0.4212	0.1444	0.4179	0.4212
	k1	k2	k3	k4
<BOS>	0.225	0.32	0.115	0.12
I	0.4993	0.3822	0.136	0.2961
am	0.4488	0.1477	0.0224	0.2649
trying	0.2911	0.0011	-0.043	0.1651
to	0.1275	-0.0665	-0.0224	0.0922
learn	0.108	0.1557	0.0734	0.069
Machine	0.2298	0.3239	0.163	0.1539
Learning	0.4179	0.4212	0.1444	0.2344
	v1	v2	v3	v4
<BOS>	0.32	0.015	0.32	0.12
I	0.4482	0.2193	0.3191	0.2961
am	0.2843	0.2389	0.0688	0.2649
trying	0.19	0.0512	-0.005	0.1651
to	0.1038	-0.0924	0.0082	0.0922
learn	0.2172	-0.1218	0.2316	0.069
Machine	0.3177	0.0201	0.3529	0.1539
Learning	0.4356	0.1684	0.3625	0.2344

Masked Score ও Softmax (দুই Head-এই, Encoder-এর মতো column split করে)

$$\text{Score} = \frac{QK^T}{\sqrt{d_k}} + M$$

Head 1 (dim 1,2) এর জন্য Masked, Scaled Score ($\text{Vd}_k = \sqrt{2} = 1.4142$) — উপরের ত্রিভুজাকার (upper-triangular, $j > i$) অংশে $-\infty$ (numerically একটি অনেক বড় ঋণাত্মক সংখ্যা, -10^9) বসানো হয়েছে:

	<BOS>	I	am	trying	to	learn	Machine	Learning
<BOS>	0.0769	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
I	0.0916	0.1717	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
am	0.0286	0.0582	0.0492	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
trying	-0.0096	-0.0112	-0.0042	0.0002	$-\infty$	$-\infty$	$-\infty$	$-\infty$
to	-0.0156	-0.0295	-0.0234	-0.0137	-0.0049	$-\infty$	$-\infty$	$-\infty$
learn	0.0414	0.0748	0.0571	0.0321	0.0106	0.02	$-\infty$	$-\infty$
Machine	0.0884	0.1584	0.1198	0.0668	0.0215	0.0427	0.09	$-\infty$
Learning	0.0997	0.1877	0.1488	0.0868	0.0312	0.0481	0.1015	0.1675

$$e^{-\infty} = 0$$

যেহেতু $e^{(-\infty)} = 0$, তাই Softmax প্রয়োগ করার পর ভবিষ্যতের token-গুলোর Attention Probability ঠিক 0 হয়ে যায় — অর্থাৎ প্রতিটি token শুধু নিজের ও আগের token-এর দিকেই মনোযোগ দিতে পারে। Head 1-এর Softmax-এর পর Attention Probability:

	<BOS>	I	am	trying	to	learn	Machine	Learning
<BOS>	1	0	0	0	0	0	0	0
I	0.48	0.52	0	0	0	0	0	0
am	0.3278	0.3376	0.3346	0	0	0	0	0
trying	0.2492	0.2487	0.2505	0.2516	0	0	0	0
to	0.2004	0.1976	0.1988	0.2007	0.2025	0	0	0
learn	0.167	0.1726	0.1696	0.1654	0.1619	0.1634	0	0
Machine	0.1434	0.1538	0.1479	0.1403	0.1341	0.137	0.1436	0
Learning	0.1237	0.1351	0.1299	0.1221	0.1155	0.1175	0.1239	0.1324

এই Probability $\times V_1(\text{dim}1,2 \text{ এর } V) = \text{head}_1 (8 \times 2)$:

	h1_1	h1_2
<BOS>	0.32	0.015
I	0.3866	0.1212
am	0.3513	0.1589
trying	0.3102	0.131
to	0.2683	0.0854
learn	0.2628	0.0545
Machine	0.2728	0.0522
Learning	0.2948	0.0686

Head 2 (dim 3,4)-এর জন্য একই প্রক্রিয়া — Masked, Scaled Score:

	<BOS>	I	am	trying	to	learn	Machine	Learning
<BOS>	0.0454	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
I	0.073	0.1281	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
am	0.049	0.0741	0.0348	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
trying	0.0238	0.0282	0.0048	-0.0087	$-\infty$	$-\infty$	$-\infty$	$-\infty$
to	0.0047	-0.0017	-0.0104	-0.0116	-0.0064	$-\infty$	$-\infty$	$-\infty$
learn	0.022	0.043	0.0309	0.0149	0.0084	0.0132	$-\infty$	$-\infty$
Machine	0.0462	0.0899	0.0643	0.0308	0.0175	0.0277	0.0617	$-\infty$
Learning	0.0697	0.1284	0.0855	0.0365	0.0209	0.0422	0.094	0.1125

Softmax-এর পর Attention Probability:

	<BOS>	I	am	trying	to	learn	Machine	Learning
<BOS>	1	0	0	0	0	0	0	0
I	0.4862	0.5138	0	0	0	0	0	0
am	0.3321	0.3405	0.3274	0	0	0	0	0
trying	0.2529	0.2541	0.2482	0.2448	0	0	0	0
to	0.202	0.2007	0.1989	0.1987	0.1997	0	0	0
learn	0.1666	0.1702	0.1681	0.1655	0.1644	0.1652	0	0
Machine	0.1425	0.1489	0.1451	0.1403	0.1385	0.1399	0.1447	0
Learning	0.1244	0.1319	0.1264	0.1204	0.1185	0.121	0.1275	0.1299

head₂ (8×2):

	h2_1	h2_2
<BOS>	0.32	0.12
I	0.3195	0.2105
am	0.2374	0.2274
trying	0.1778	0.2117
to	0.143	0.1876
learn	0.158	0.1688
Machine	0.187	0.1675
Learning	0.2111	0.1767

Concatenation ও Output Projection

head₁ এবং head₂ জোড়া দিয়ে (8×4):

	h1_1	h1_2	h2_1	h2_2
<BOS>	0.32	0.015	0.32	0.12
I	0.3866	0.1212	0.3195	0.2105
am	0.3513	0.1589	0.2374	0.2274
trying	0.3102	0.131	0.1778	0.2117
to	0.2683	0.0854	0.143	0.1876
learn	0.2628	0.0545	0.158	0.1688
Machine	0.2728	0.0522	0.187	0.1675
Learning	0.2948	0.0686	0.2111	0.1767

W_O (Decoder self-attention-এর নিজস্ব) দিয়ে গুণ করে চূড়ান্ত Masked MultiHeadOutput (8×4):

	dim1	dim2	dim3	dim4
<BOS>	0.047	0.096	0.1095	0.059
I	0.084	0.1093	0.1357	0.0983
am	0.0896	0.094	0.1212	0.101
trying	0.0784	0.0798	0.1009	0.0863
to	0.0627	0.068	0.0827	0.0689
learn	0.0541	0.0683	0.0802	0.0605
Machine	0.0545	0.0733	0.0866	0.0626
Learning	0.0609	0.0801	0.0962	0.0702

STEP 13: Add & Norm

X_dec-এর সাথে MHod যোগ (Residual):

	dim1	dim2	dim3	dim4
<BOS>	0.097	1.146	0.1595	1.109
I	1.0254	0.8496	0.4457	1.4982
am	1.1489	-0.2221	0.3912	1.1508
trying	0.2695	-0.6102	0.2309	1.2859
to	-0.4941	-0.5357	0.2727	1.1681
learn	-0.6049	0.602	0.1802	1.2092
Machine	0.0251	1.1834	0.3466	1.1108
Learning	0.8178	1.184	0.2662	1.3177

এর উপর Layer Normalization প্রয়োগ করলে ($\gamma=1, \beta=0$):

- <BOS>: $\mu = 0.6279, \sigma^2 = 0.2503$
- I: $\mu = 0.9548, \sigma^2 = 0.1426$
- am: $\mu = 0.6172, \sigma^2 = 0.3308$
- trying: $\mu = 0.2940, \sigma^2 = 0.4515$
- to: $\mu = 0.1028, \sigma^2 = 0.4819$
- learn: $\mu = 0.3466, \sigma^2 = 0.4356$
- Machine: $\mu = 0.6665, \sigma^2 = 0.2446$
- Learning: $\mu = 0.8964, \sigma^2 = 0.1659$

চূড়ান্ত LayerNorm আউটপুট LN3 (8×4) — এটিই Decoder-এর নিজস্ব representation, যা এখন Cross-Attention-এ Query হিসেবে ব্যবহৃত হবে:

	dim1	dim2	dim3	dim4
<BOS>	-1.0611	1.0356	-0.9362	0.9617
I	0.1871	-0.2785	-1.3477	1.439
am	0.9246	-1.4594	-0.3929	0.9278
trying	-0.0365	-1.3457	-0.094	1.4761
to	-0.8598	-0.9196	0.2448	1.5346
learn	-1.4417	0.3869	-0.2522	1.3069
Machine	-1.2969	1.0452	-0.6467	0.8984
Learning	-0.1929	0.706	-1.5474	1.0343

STEP 14: Cross-Attention

Purpose ও পার্থক্য

Cross-Attention, Self-Attention থেকে আলাদা একটি জায়গায়: Query আসে Decoder থেকে, কিন্তু Key ও Value আসে Encoder-এর চূড়ান্ত Output (H_{enc}) থেকে। এভাবে Decoder প্রতিটি ধাপে সরাসরি Input বাক্যের (Encoder-এর প্রসেস করা) প্রাসঙ্গিক তথ্য মনোযোগ দিতে পারে।

$$Q = Y_{dec} W_Q$$

$$K = H_{enc} W_K$$

$$V = H_{enc} W_V$$

লক্ষ্য করুন: Q তৈরি হয় LN3 (Decoder, 8×4) থেকে, কিন্তু K এবং V তৈরি হয় H_{enc} (Encoder Output, 7×4) থেকে — তাই Q-এর সারি সংখ্যা (8) এবং K, V-এর সারি সংখ্যা (7) ভিন্ন হতে পারে, এটাই স্বাভাবিক।

Q ($LN3 \times W_Q$, আকার 8×4):

	q1	q2	q3	q4
<BOS>	0.1972	-0.1972	0.2023	-0.0125
I	0.1069	-0.1069	0.1252	0.1535
am	-0.1067	0.1067	0.0003	0.1317
trying	-0.1252	0.1252	0.1513	0.0058
to	-0.1164	0.1164	0.2394	-0.1105
learn	0.0639	-0.0639	0.2749	-0.1189
Machine	0.1692	-0.1692	0.2195	-0.065
Learning	0.2253	-0.2253	0.1227	0.1354

K ($H_{enc} \times W_K$, আকার 7×4):

	k1	k2	k3	k4
I	-0.0283	0.0781	-0.166	0.2007
am	0.2295	-0.2057	-0.1248	0.2528
trying	0.156	-0.3084	0.0295	0.206
to	0.0288	-0.3024	0.1407	0.1399
learn	-0.073	-0.1535	0.06	0.1777
Machine	-0.1436	0.0554	-0.0388	0.1345
Learning	-0.0466	0.1311	-0.1784	0.1692

V ($H_{enc} \times W_V$, আকার 7×4):

	v1	v2	v3	v4
I	-0.166	-0.0283	-0.0348	0.229
am	-0.1248	0.2295	-0.1279	0.0233
trying	0.0295	0.156	-0.2355	0.05
to	0.1407	0.0288	-0.2806	0.1111
learn	0.06	-0.073	-0.2377	0.2507
Machine	-0.0388	-0.1436	-0.0957	0.2781
Learning	-0.1784	-0.0466	0.0092	0.2159

Attention Score, Scale, Softmax

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

QK^T এখানে $(8 \times 4) \cdot (4 \times 7) = (8 \times 7)$ আকারের একটি ম্যাট্রিক্স দেয় (৮টি Decoder position, প্রতিটি ৭টি Encoder token-কে কতটা গুরুত্ব দিচ্ছে)। $d_k = 4$ (পুরো dimension ব্যবহৃত হচ্ছে, এখানে Head ভাগ করা হয়নি — single-head cross-attention হিসেবে দেখানো হলো সরলতার জন্য), তাই $\sqrt{d_k} = 2$ । Scaled Score (8×7) :

	I	am	trying	to	learn	Machine	Learning
<BOS>	-0.0285	0.0287	0.0475	0.046	0.0129	-0.0244	-0.0366
I	-0.0007	0.0348	0.0425	0.0372	0.0217	-0.0027	-0.0077
am	0.0189	-0.0066	-0.0112	-0.0084	0.0074	0.0195	0.0206
trying	-0.0053	-0.0359	-0.0262	-0.0097	0	0.0099	-0.0019
to	-0.0248	-0.0542	-0.0349	-0.0102	-0.0073	-0.0005	-0.0204
learn	-0.0381	-0.0183	0.0067	0.0216	0.0002	-0.0197	-0.0403
Machine	-0.0337	0.0149	0.0358	0.0389	0.0076	-0.0255	-0.0401
Learning	-0.0086	0.0585	0.0681	0.0554	0.0248	-0.0157	-0.0195

প্রতিটি সারিতে (৮টি Decoder position-এর প্রতিটির জন্য) Softmax প্রয়োগ করলে Cross-Attention Probability (8×7) , প্রতিটি সারির যোগফল 1:

	I	am	trying	to	learn	Machine	Learning
<BOS>	0.1379	0.146	0.1487	0.1485	0.1437	0.1384	0.1367
I	0.1402	0.1453	0.1464	0.1456	0.1434	0.1399	0.1392
am	0.1447	0.1411	0.1404	0.1408	0.1431	0.1448	0.145
trying	0.1435	0.1392	0.1405	0.1429	0.1443	0.1457	0.144
to	0.1424	0.1383	0.141	0.1445	0.1449	0.1459	0.143
learn	0.1392	0.142	0.1456	0.1478	0.1447	0.1418	0.1389
Machine	0.1381	0.145	0.148	0.1485	0.1439	0.1392	0.1372
Learning	0.1383	0.1479	0.1493	0.1474	0.143	0.1373	0.1368

Full Numerical Calculation — Weighted Values

Attention Probability $(8 \times 7) \times V (7 \times 4) =$ Cross-Attention Output (8×4) — এভাবেই প্রতিটি Decoder position, Encoder-এর তথ্যের একটি ওজনযুক্ত সমষ্টি (weighted sum) পায়:

	dim1	dim2	dim3	dim4
<BOS>	-0.037	0.0203	-0.1463	0.1629
I	-0.0383	0.0193	-0.145	0.1639
am	-0.0406	0.0162	-0.1422	0.1665
trying	-0.0396	0.0157	-0.1429	0.1668
to	-0.0389	0.0157	-0.1435	0.1667
learn	-0.0374	0.0182	-0.1454	0.1646
Machine	-0.037	0.0198	-0.1461	0.1633
Learning	-0.0374	0.021	-0.1461	0.1625

STEP 15: Add & Norm

LN3-এর সাথে Cross-Attention Output যোগ (Residual):

	dim1	dim2	dim3	dim4
<BOS>	-1.0981	1.056	-1.0825	1.1246
I	0.1489	-0.2591	-1.4927	1.6029
am	0.884	-1.4432	-0.5351	1.0943
trying	-0.0761	-1.3299	-0.2369	1.6429
to	-0.8987	-0.904	0.1014	1.7013
learn	-1.479	0.4052	-0.3976	1.4715
Machine	-1.3339	1.065	-0.7929	1.0618
Learning	-0.2303	0.727	-1.6935	1.1968

Layer Normalization-এর পর ($\gamma=1$, $\beta=0$):

- <BOS>: $\mu = 0.0000$, $\sigma^2 = 1.1894$
- I: $\mu = 0.0000$, $\sigma^2 = 1.2217$
- am: $\mu = 0.0000$, $\sigma^2 = 1.0870$
- trying: $\mu = 0.0000$, $\sigma^2 = 1.1324$
- to: $\mu = 0.0000$, $\sigma^2 = 1.1324$
- learn: $\mu = 0.0000$, $\sigma^2 = 1.1688$
- Machine: $\mu = 0.0000$, $\sigma^2 = 1.1674$
- Learning: $\mu = 0.0000$, $\sigma^2 = 1.2205$

চূড়ান্ত LN4 (8×4):

	dim1	dim2	dim3	dim4
<BOS>	-1.0069	0.9683	-0.9926	1.0312
I	0.1347	-0.2344	-1.3505	1.4502
am	0.8479	-1.3842	-0.5132	1.0496
trying	-0.0715	-1.2497	-0.2226	1.5438
to	-0.8445	-0.8495	0.0953	1.5988
learn	-1.3681	0.3748	-0.3678	1.3611
Machine	-1.2346	0.9857	-0.7338	0.9827
Learning	-0.2085	0.6581	-1.5329	1.0833

STEP 16: Feed Forward Network (Decoder)

Encoder-এর FFN-এর মতোই একই সূত্র ও একই W_1 , b_1 , W_2 , b_2 (এই নোটে সরলতার জন্য একই মান পুনরায় ব্যবহার করা হয়েছে, বাস্তবে Decoder-এর নিজস্ব আলাদা FFN Weight থাকে):

$$FFN(x) = W_2 \text{ReLU}(W_1 x + b_1) + b_2$$

প্রতিটি ৮টি Decoder position-এর জন্য $W_1 x + b_1$ গণনা করে সম্পূর্ণ z_2 (8×4):

	z1	z2	z3	z4
<BOS>	0.3048	-0.2048	0.2951	-0.1951
I	0.5386	-0.4386	-0.0954	0.1954
am	0.446	-0.346	-0.313	0.413
trying	0.4239	-0.3239	-0.1651	0.2651
to	0.3258	-0.2258	0.0241	0.0759
learn	0.2722	-0.1722	0.275	-0.175
Machine	0.2465	-0.1465	0.3472	-0.2472

	z1	z2	z3	z4
Learning	0.4491	-0.3491	0.0992	0.0008

ReLU প্রয়োগের পর (8×4):

	a1	a2	a3	a4
<BOS>	0.3048	0	0.2951	0
I	0.5386	0	0	0.1954
am	0.446	0	0	0.413
trying	0.4239	0	0	0.2651
to	0.3258	0	0.0241	0.0759
learn	0.2722	0	0.275	0
Machine	0.2465	0	0.3472	0
Learning	0.4491	0	0.0992	0.0008

W2·a+b2 প্রয়োগ করে চূড়ান্ত FFN Output (8×4):

	f1	f2	f3	f4
<BOS>	0.0305	0.1905	0.059	0.1305
I	0.0734	0.2077	0.0391	0.1734
am	0.0859	0.1892	0.0826	0.1859
trying	0.0689	0.1848	0.053	0.1689
to	0.0402	0.1676	0.02	0.1402
learn	0.0272	0.1819	0.055	0.1272
Machine	0.0246	0.184	0.0694	0.1246
Learning	0.045	0.1997	0.02	0.145

STEP 17: Add & Norm (চূড়ান্ত)

LN4-এর সাথে FFN Output যোগ (Residual):

	dim1	dim2	dim3	dim4
<BOS>	-0.9764	1.1587	-0.9336	1.1617
I	0.2081	-0.0267	-1.3114	1.6236
am	0.9338	-1.195	-0.4306	1.2355
trying	-0.0026	-1.065	-0.1696	1.7127
to	-0.8043	-0.6819	0.1153	1.7389
learn	-1.3408	0.5567	-0.3128	1.4883
Machine	-1.2099	1.1697	-0.6644	1.1073
Learning	-0.1635	0.8578	-1.5129	1.2283

চূড়ান্ত Layer Normalization প্রয়োগের পর:

- <BOS>: $\mu = 0.1026$, $\sigma^2 = 1.1187$
- I: $\mu = 0.1234$, $\sigma^2 = 1.0847$
- am: $\mu = 0.1359$, $\sigma^2 = 0.9845$
- trying: $\mu = 0.1189$, $\sigma^2 = 1.0100$
- to: $\mu = 0.0920$, $\sigma^2 = 1.0288$
- learn: $\mu = 0.0978$, $\sigma^2 = 1.0956$
- Machine: $\mu = 0.1007$, $\sigma^2 = 1.1148$
- Learning: $\mu = 0.1024$, $\sigma^2 = 1.1296$

এটিই চূড়ান্ত Decoder Representation Y_{dec} (8×4) — একটি সম্পূর্ণ Decoder Layer-এর আউটপুট:

	dim1	dim2	dim3	dim4
<BOS>	-1.0201	0.9985	-0.9796	1.0013
I	0.0813	-0.1441	-1.3776	1.4404
am	0.8041	-1.3413	-0.571	1.1082
trying	-0.1209	-1.178	-0.2871	1.586
to	-0.8837	-0.763	0.0229	1.6237
learn	-1.3745	0.4384	-0.3923	1.3284
Machine	-1.2413	1.0125	-0.7246	0.9534
Learning	-0.2502	0.7107	-1.5199	1.0594

এখন আমরা শেষ position ("Learning" দেখার পর, পরবর্তী token অনুমানের জন্য দায়ী position)-এর ভেক্টরটি ব্যবহার করে Next-Token Prediction করব।

অংশ ৬: Output Generation

STEP 18: Linear Layer

Purpose

Decoder-এর চূড়ান্ত Representation ($d_{\text{model}} = 4$ dimension-এর একটি ভেক্টর) সরাসরি কোনো Probability নয় — এটি এখনও একটি abstract vector। এই ভেক্টরকে Vocabulary-এর প্রতিটি সম্ভাব্য শব্দের সাথে তুলনীয় একটি score (Logit)-এ রূপান্তর করতে হয়। এই কাজটি করে একটি Linear (Fully Connected) Layer, যা d_{model} dimension থেকে Vocabulary size (V) dimension-এ map করে।

এই নোটে হাতে-কলমে গণনা করার জন্য একটি ছোট Vocabulary ($V = 10$) ব্যবহার করা হচ্ছে:

0:<EOS> 1:the 2:is 3:Machine 4:Learning 5:learn 6:a 7:to 8:and 9:I

$$z = YW_L + b_L$$

$W_L \in \mathbb{R}^{4 \times 10}$, $b_L \in \mathbb{R}^{10}$:

	<EOS>	the	is	Machine	Learning	learn	a	to	and	I
row1	0.1	0.2	0	0.1	0.2	0	0.1	0	0.2	0.1
row2	0.2	0	0.1	0.2	0	0.1	0	0.2	0.1	0
row3	0	0.1	0.2	0	0.1	0.2	0.1	0	0	0.2
row4	0.1	0	0.1	0.1	0.2	0	0.2	0.1	0.1	0

$b_L = [0.0500, 0.0000, 0.0500, 0.1000, 0.0000, 0.0500, 0.0000, 0.1000, 0.0500, 0.0000]$

Decoder-এর শেষ position ("Learning" এর পরের position)-এর ভেক্টর $y = [-0.2502, 0.7107, -1.5199, 1.0594]$ । এই y কে W_L দিয়ে গুণ করে b_L যোগ করলে ১০টি Logit পাওয়া যায়:

$z = y \cdot W_L + b_L = [0.2731, -0.2020, -0.0770, 0.3231, 0.0098, -0.1829, 0.0349, 0.3481, 0.1770, -0.3290]$

STEP 19: Softmax

$$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

প্রতিটি Logit-এর উপর exponential নেওয়া হলো:

$e^z = [0.9277, 0.5769, 0.6537, 0.9753, 0.7130, 0.5880, 0.7311, 1.0000, 0.8427, 0.5081]$

যোগফল $\sum e^z = 7.5166$

প্রতিটি e^z -কে এই যোগফল দিয়ে ভাগ করলে চূড়ান্ত Probability পাওয়া যায়। নিচে সম্পূর্ণ টেবিলে সবগুলো Token, Logit, Probability এবং Percentage দেখানো হলো:

	Logit	Probability	Percentage
<EOS>	0.2731	0.1234	12.34%
the	-0.2020	0.0767	7.67%
is	-0.0770	0.0870	8.70%
Machine	0.3231	0.1298	12.98%
Learning	0.0098	0.0949	9.49%
learn	-0.1829	0.0782	7.82%
a	0.0349	0.0973	9.73%
to	0.3481	0.1330	13.30%
and	0.1770	0.1121	11.21%

	Logit	Probability	Percentage
I	-0.3290	0.0676	6.76%

যাচাই: $\sum P_i = 1.0000 \approx 1 \checkmark$

STEP 20: Next Token

$$\hat{y} = \arg \max_i P_i$$

সবচেয়ে বেশি Probability-যুক্ত token হলো "to" (Probability = 0.1330, অর্থাৎ 13.30%)। এটিই মডেলের পরবর্তী token হিসেবে predict করা token।

(লক্ষণীয়: যেহেতু এই নোটের সব Weight Matrix ছোট, কাল্পনিক সংখ্যা — কোনো প্রকৃত Training হয়নি — তাই এই predicted token ভাষাগতভাবে অর্থপূর্ণ নাও হতে পারে। বাস্তব Transformer-এ W_E , W_Q , W_K , W_V , W_L ইত্যাদি বিশাল ডেটাসেটে Train করার পর, এই একই গাণিতিক প্রক্রিয়া দিয়ে ভাষাগতভাবে সঠিক token predict হয়।)

Inference-এর সময় predicted token-টি Decoder Input-এর শেষে যোগ করে দেওয়া হয় (যেমন $\langle \text{BOS} \rangle$, I, am, ..., Learning, [predicted token]), এবং সম্পূর্ণ Decoder পাস (STEP 9 থেকে STEP 20) আবার চালানো হয় পরবর্তী token predict করার জন্য। এই প্রক্রিয়া চলতে থাকে যতক্ষণ না $\langle \text{EOS} \rangle$ token predict হয়।

অংশ ৭: Training

Training-এর সময় সম্পূর্ণ প্রক্রিয়া নিম্নরূপ ধারাবাহিকভাবে ঘটে:

Forward Pass → Logits → Softmax → Loss → Backpropagation → Optimizer → Weight Update

প্রতিটি ধাপ যা আমরা এতক্ষণ গণনা করেছি (Embedding থেকে Softmax পর্যন্ত) মিলিয়েই একটি Forward Pass। এরপর প্রকৃত Target token-এর সাথে Predicted Probability তুলনা করে Loss (ভুলের পরিমাণ) গণনা করা হয় — সাধারণত Cross-Entropy Loss ব্যবহার করে:

$$L = - \sum_i y_i \log(p_i)$$

এখানে y_i হলো প্রকৃত (True) Label — One-Hot আকারে (সঠিক token-এর জন্য 1, বাকি সবার জন্য 0), এবং p_i হলো মডেলের predict করা Probability। যদি মডেল সঠিক token-কে খুব কম Probability দেয়, তাহলে Loss অনেক বড় হয়।

এরপর Backpropagation Algorithm ব্যবহার করে এই Loss-এর সাপেক্ষে প্রতিটি Weight Matrix-এর Gradient (আংশিক অন্তরক, partial derivative) গণনা করা হয়, এবং একটি Optimizer (যেমন Adam) সেই Gradient ব্যবহার করে প্রতিটি Weight সামান্য পরিমাণে আপডেট করে, যাতে পরের বার Loss কিছুটা কমে। যেসব Parameter এভাবে শেখা (learn) হয়:

$$W_E, W_Q, W_K, W_V, W_O, W_1, W_2, W_L$$

লক্ষ লক্ষ বাক্যের উপর এই প্রক্রিয়া বারবার (many epochs) চালানো হলে, এই Weight-গুলো ধীরে ধীরে এমন মান নেয় যা প্রকৃত ভাষার প্যাটার্ন প্রতিফলিত করে — তখনই মডেল অর্থপূর্ণ Text Generate করতে সক্ষম হয়।

অংশ ৮: Inference

- Training শেষ হওয়ার পর সব Weight Matrix ($W_E, W_Q, W_K, W_V, W_O, W_1, W_2, W_L$ ইত্যাদি) স্থির (Frozen) হয়ে যায় — Inference-এর সময় এগুলো আর পরিবর্তিত হয় না।
- Decoder একবারে একটি করে (one-token-at-a-time) token Generate করে — প্রতিটি ধাপে STEP 18–20-এর মতো Softmax থেকে সবচেয়ে বেশি Probability-যুক্ত token বেছে নেওয়া হয় (অথবা Sampling ব্যবহার করা হয়)।
- প্রতিটি নতুন Generate করা token পরের ধাপে Decoder-এর Input-এর শেষে যোগ হয়ে যায়, এবং সম্পূর্ণ Decoder পাস আবার চালানো হয়।
- এই প্রক্রিয়া চলতে থাকে যতক্ষণ না মডেল <EOS> (End of Sentence) token Generate করে, অথবা একটি নির্দিষ্ট সর্বোচ্চ length-এ পৌঁছে যায়।

অংশ ৯: BERT বনাম GPT

BERT

Encoder-only

- শুধুমাত্র Encoder ব্যবহার করে (Encoder-only architecture)।
- Bidirectional Attention — প্রতিটি token বাক্যের আগে ও পরে, উভয় দিকের token-ই দেখতে পারে (কোনো Causal Mask নেই)।
- Masked Language Modeling (MLM)-এর মাধ্যমে Train করা হয় — বাক্যের কিছু token লুকিয়ে (mask করে) মডেলকে সেগুলো অনুমান করতে শেখানো হয়।
- মূলত Language Understanding-এর কাজে ব্যবহৃত হয় — যেমন Classification, Question Answering, Sentiment Analysis।

GPT

Decoder-only

- শুধুমাত্র Decoder ব্যবহার করে (Decoder-only architecture)।
- Causal (Masked) Attention — এই নোটের STEP 12-এ যেমন দেখানো হয়েছে, প্রতিটি token শুধু আগের token দেখতে পারে।
- Autoregressive Next-Token Prediction-এর মাধ্যমে Train করা হয় — ঠিক যেভাবে STEP 18–20-এ দেখানো হয়েছে।
- মূলত Text Generation-এর কাজে ব্যবহৃত হয় — যেমন Chatbot, Writing Assistant, Code Generation।

দুটোই মূল Transformer আর্কিটেকচার থেকে এসেছে: BERT মূল Transformer-এর শুধু Encoder অংশ নেয়, আর GPT শুধু Decoder অংশ নেয় (Cross-Attention ছাড়া, কারণ এখানে আলাদা কোনো Encoder নেই)। এই নোটে আমরা যে সম্পূর্ণ Encoder-Decoder গণনা করেছি, তা থেকেই এই দুটি সরলীকৃত (simplified) সংস্করণ বোঝা সহজ।

চূড়ান্ত সেকশন: সম্পূর্ণ গাণিতিক Cheat Sheet

Transformer-এর সব প্রধান সূত্র এক জায়গায় — Input থেকে Output পর্যন্ত ক্রম অনুসারে।

Embedding

$$e_i = \text{OneHot}(t_i) W_E$$

Positional Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Position-aware Representation

$$X = E + PE$$

Query, Key, Value

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Masked Attention

$$\text{Score} = \frac{QK^T}{\sqrt{d_k}} + M$$

Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

Feed Forward Network

$$FFN(x) = W_2 \text{ReLU}(W_1 x + b_1) + b_2$$

Layer Normalization

$$\text{LayerNorm}(x_i) = \gamma \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Linear

$$z = YW_L + b_L$$

Softmax

$$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Next Token

$$\hat{y} = \arg \max_i P_i$$

Cross-Entropy Loss (Training)

$$L = - \sum_i y_i \log(p_i)$$